Mini Juego Piedra, Papel o Tijera con Persistencia: Struct y Flujos de Archivo para Memoria Volátil y Disco No Volátil

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B — Grupo A

Prof. Mansilla Muñoz York Elías

2026 — Grupo A

Resumen

Se desarrolla un juego de Piedra, Papel o Tijera contra inteligencia artificial donde cada partida se modela con la estructura *Partida* y se persiste en disco con flujo de salida en modo adjunto (*ios::app*), recuperándose con flujo de entrada en formato CSV. Se contrasta memoria RAM volátil con disco no volátil. Las pruebas demuestran que el historial persiste tras cerrar el ejecutable. Se incluyen las cinco preguntas del cuestionario, desarrolladas en formato APA 7 sin sección de Referencias por no ser obligatoria.

Palabras clave: persistencia, fstream, ofstream, ifstream, ios::app, struct, memoria volátil, disco no volátil

Abstract

A Rock, Paper, Scissors game is developed with the *Partida* structure persisted to disk with append mode and recovered via input stream. Volatile RAM versus non-volatile disk is contrasted and a five-item questionnaire is included, formatted in APA 7 without References.

Keywords: persistence, fstream, struct

Introducción

Muchos programadores confunden RAM volátil con disco no volátil. La Actividad 7 exige implementar ambas: la partida reside en RAM como estructura y luego se escribe en disco para sobrevivir al cierre. Los objetivos son modelar con estructura, escribir con flujo de salida en modo adjunto sin sobrescribir y leer con flujo de entrada. El juego es el vehículo; la persistencia es el aprendizaje.

Revisión de la Literatura

La biblioteca <*fstream*> gestiona flujos de archivo; el flujo de salida escribe al disco y el de entrada lee desde él. La bandera *ios::app* indica adjuntar al final sin truncar, y

cin.ignore() limpia el salto de línea antes de *getline*. La estructura agrupa datos heterogéneos en una sola entidad.

Flujo	Dirección	Modo	Efecto sin modo
ofstream	RAM → Disco	ios::app	Sin app trunca y borra
ifstream	Disco → RAM	ios::in	Lee CSV secuencial

Nota. Flujos y modos.

Método

La estructura */piedrapapelotijera* contiene *src/main.cpp* y *src/historial_partidas.txt*. La función *guardarEnDisco* abre *ofstream("src/historial_partidas.txt", ios::app)* y escribe CSV; *leerHistorialDisco* abre *ifstream* y lee con *getline* delimitado. La compilación se ejecutó con *g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe*. Las pruebas consistieron en jugar una partida, cerrar la consola, reabrir y seleccionar la opción 2 para verificar la no volatilidad.

Resultados

Tras tres partidas el archivo contuvo tres líneas CSV. Cerrar y reabrir el ejecutable no eliminó el archivo; la opción 2 listó las tres partidas. Sin *ios::app* solo permanecería la última. La captura *capturas/jugada_consola.png* muestra el menú y la partida; *capturas/archivo_disco.png* muestra el archivo en disco.

Cuestionario Técnico de Control

Se responden las cinco preguntas de la guía de la Actividad 7, en formato APA 7.

Pregunta 1 — RAM vs. Disco

Enunciado: ¿Por qué al cerrar y reabrir el ejecutable la Opción 2 sigue mostrando historial?

Respuesta: Porque el historial no reside en RAM volátil —que se borra al cerrar— sino en disco no volátil en *historial_partidas.txt*. Al reabrir, el flujo de entrada lee el archivo persistido; la RAM solo lo carga temporalmente para impresión. Esta es la

diferencia entre memoria de trabajo y almacenamiento secundario. La prueba de cerrar y reabrir demuestra la no volatilidad.

Pregunta 2 — Uso de ios::app

Enunciado: ¿Qué pasaría si guardarEnDisco usa ofstream sin ios::app?

Respuesta: Abriría en modo truncado por defecto y borraría el contenido previo; cada partida sobrescribiría la anterior y solo quedaría la última, con pérdida del historial. Con *ios::app* el puntero se posiciona al final y adjunta sin borrar, preservando todas las partidas. Es la bandera de adjuntado que evita el truncado.

Pregunta 3 — Manejo del Búfer

Enunciado: ¿Por qué es necesario cin.ignore() antes de getline del nombre?

Respuesta: Porque tras *cin >> opcion* queda el salto de línea en el búfer; sin *ignore(), getline* lo consume como línea vacía y se salta la pregunta de apodo, dejando el campo vacío. Es el defecto clásico al combinar >> con *getline*. La llamada a *cin.ignore()* descarta ese carácter y permite la lectura correcta, como se documenta en la guía.

Pregunta 4 — Agrupación con struct

Enunciado: ¿Qué ventajas ofrece struct Partida frente a variables sueltas?

Respuesta: Cohesión: agrupa datos de una partida —identificador, jugador, elecciones y resultado— en una sola entidad, facilita pasar una sola *Partida* a funciones como *guardarEnDisco* y *leerHistorialDisco*, mejora legibilidad y evita desincronizar parámetros sueltos. Una sola estructura representa una jugada completa, con tipado y manejo coherente, frente a cinco variables sueltas que deben pasarse y sincronizarse manualmente.

Pregunta 5 — Aplicación al PIA

Enunciado: ¿Qué datos del PIA guardarías con esta técnica?

Respuesta: Respuesta personal del estudiante —Grupo A—: se guardarían registros de sensores del prototipo de Expotécnica con *timestamp, idSensor, temperatura, humedad, estado* en formato CSV, una línea por lectura, para posterior graficado y auditoría. Cada lectura sería una estructura análoga a *Partida* pero con campos de sensor, persistida con el mismo patrón *ofstream* con *ios::app* e *ifstream* para recuperación.

Discusión

La persistencia en disco explica la supervivencia del historial; el modo adjunto preserva el contenido y el búfer debe gestionarse para la entrada de texto. El patrón estructura y flujo es aplicable a registros del Proyecto Integrador Anual y sienta bases para la programación orientada a objetos.

Conclusión

El juego quedó funcional con persistencia verificada y el cuestionario de cinco preguntas respondido en formato APA 7. Como trabajo futuro se propone migrar de CSV a JSON o SQLite y validar archivo corrupto. La técnica es directamente reutilizable para el PIA.